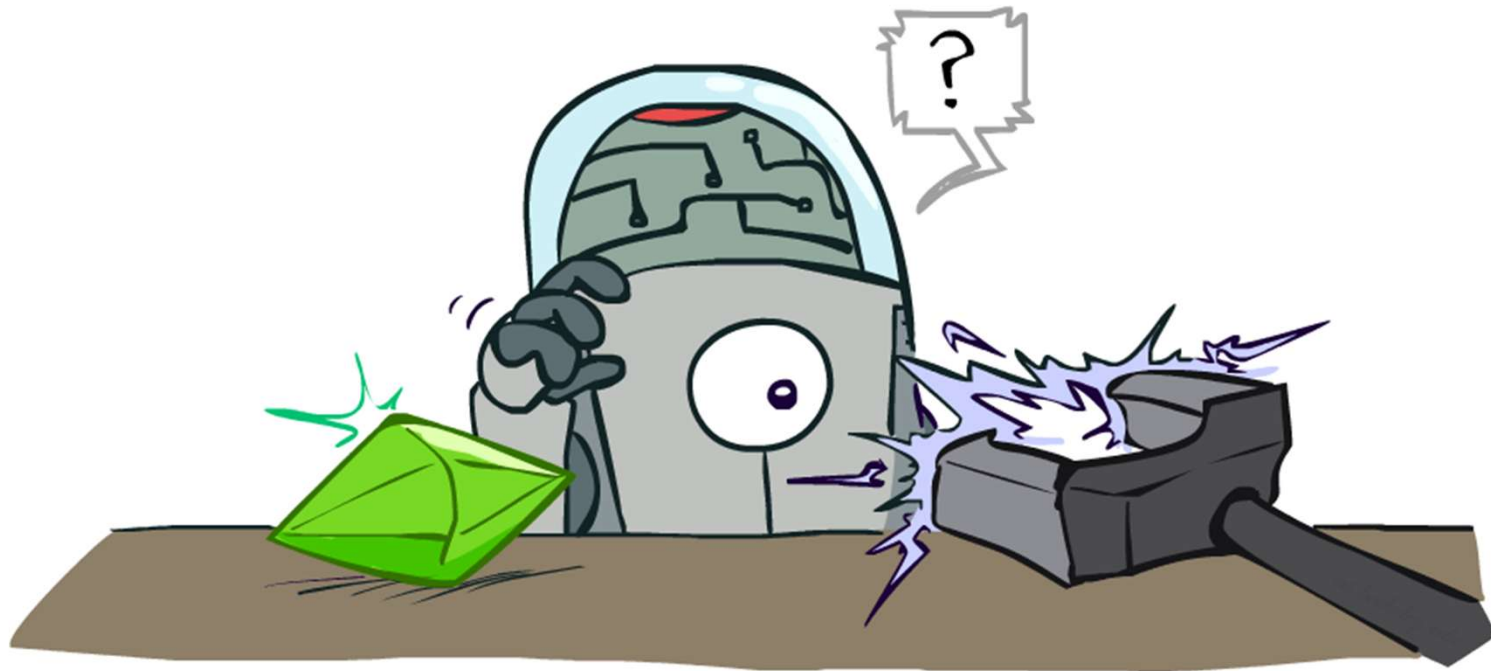


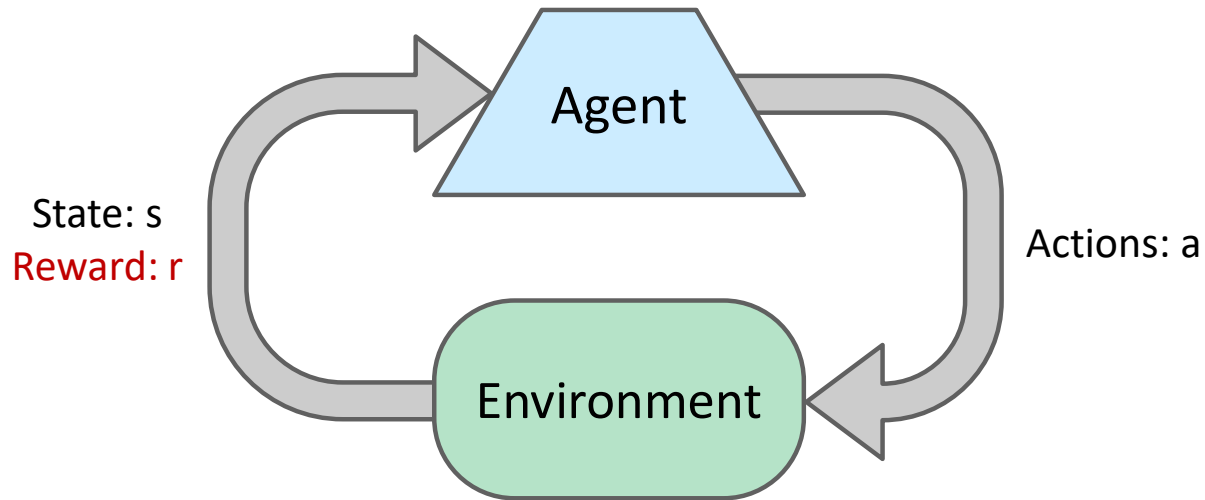
Reinforcement Learning



KNTU, Spring 2025

Slides are borrowed from UC Berkeley

Reinforcement Learning



- Basic idea:
 - Receive feedback in the form of **rewards**
 - Agent's utility is defined by the reward function
 - Must (learn to) act so as to **maximize expected rewards**
 - All learning is based on observed samples of outcomes!

Reinforcement Learning

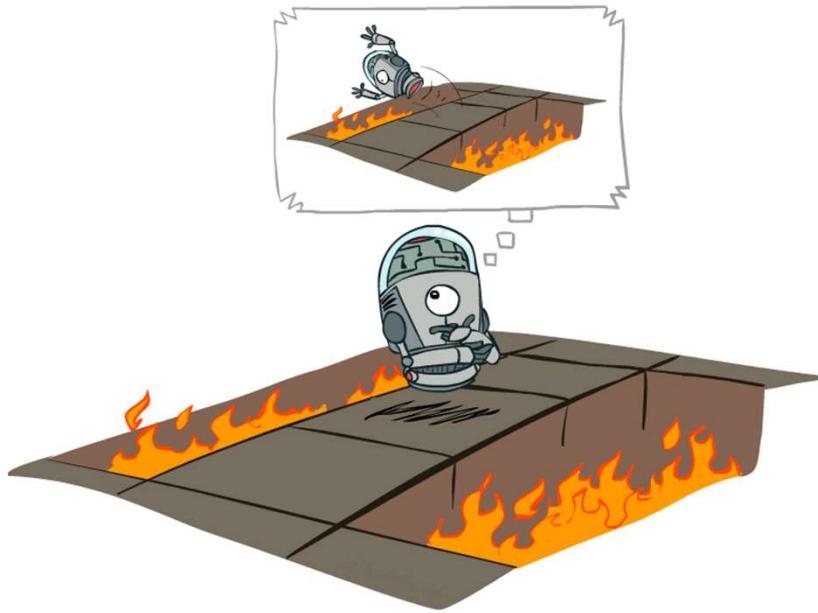
- Still assume a Markov decision process (MDP):
 - A set of states $s \in S$
 - A set of actions (per state) $A(s)$
 - A transition model $T(s,a,s')$
 - A reward function $R(s,a,s')$
- Still looking for a policy $\pi(s)$
- New twist: don't know T or R
 - I.e. we don't know which states are good or what the actions do
 - Must explore new states and actions to discover how the world works



Reinforcement Learning

- What if the MDP is initially unknown? Lots of things change!
 - **Exploration**: you have to *try unknown actions* to get information
 - **Exploitation**: eventually, you have to use what you know
 - **Regret**: early on, you inevitably “make mistakes” and lose reward
 - **Sampling**: you may need to repeat many times to get good estimates
 - **Generalization**: what you learn in one state may apply to others too

Offline (MDPs) vs. Online (RL)



Offline Solution

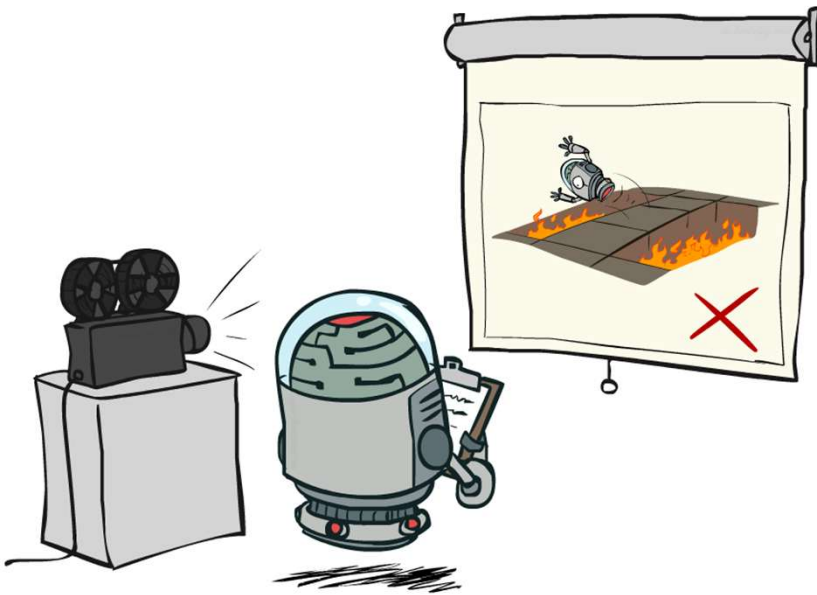


Online Learning

Approaches to reinforcement learning

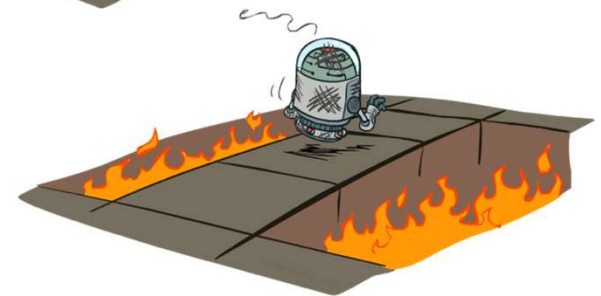
1. Model-based: Learn the model, solve it, execute the solution
2. Learn values from experiences, use to make decisions
 - a. Direct evaluation (Monte Carlo learning)
 - b. Temporal difference learning
 - c. SARSA
 - d. Q-learning
3. Optimize the policy directly

Passive vs Active Reinforcement Learning

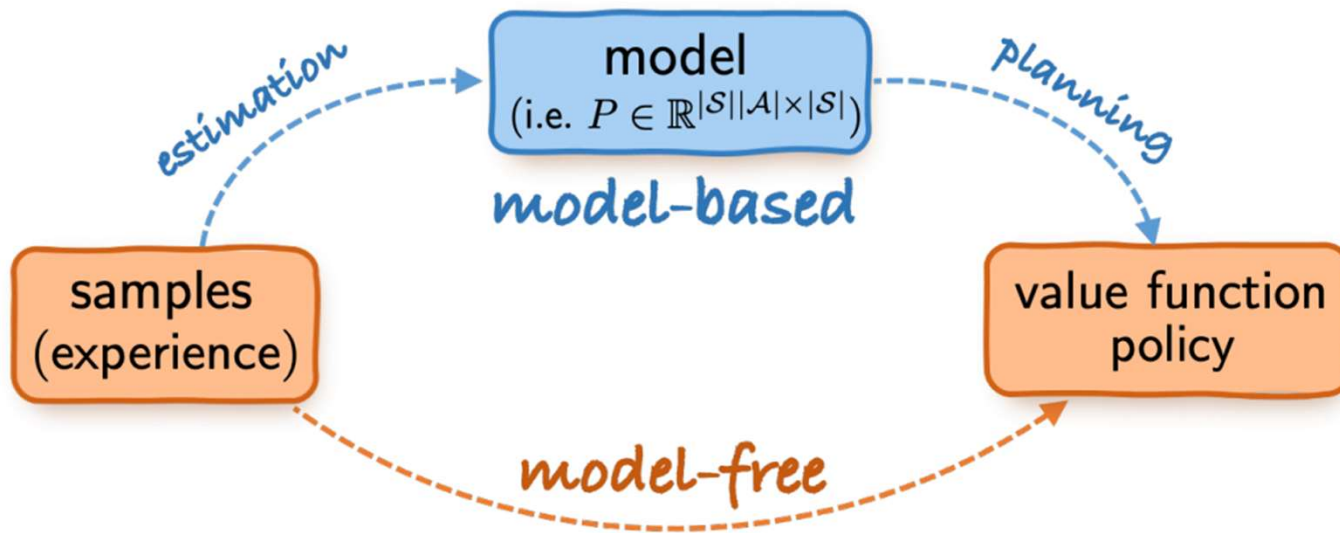
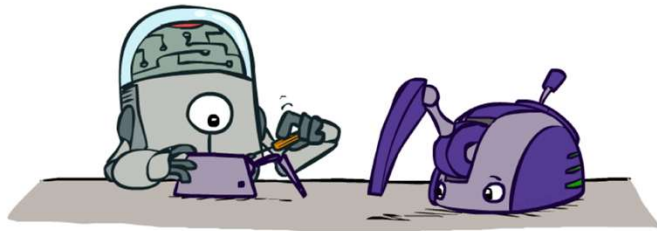


passively observing or following a predefined policy

- ✓ The agent **chooses** actions and updates its policy to improve future decisions
- ✓ agent **actively improves** its policy

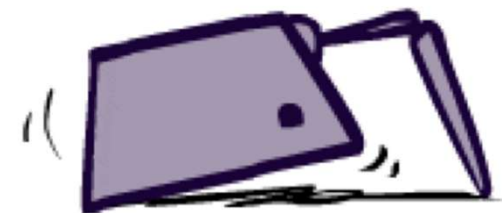


Model-Based RL



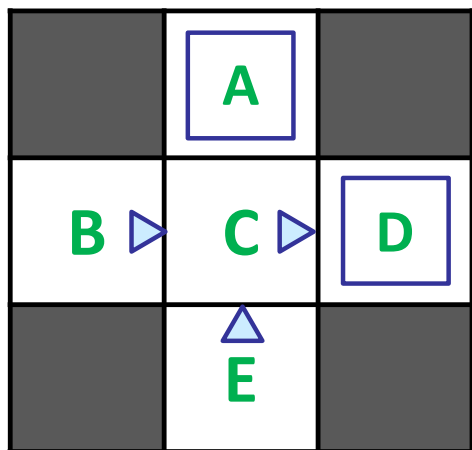
Model-Based Learning

- Model-Based Idea:
 - Learn an **approximate model** based on experiences
 - Sample trajectories according to given policy
 - **Solve** for values as if the learned model were correct
- Step 1: Learn empirical MDP model
 - **Count** outcomes s' for each s, a
 - Directly estimate each entry in $T(s, a, s')$ from counts
 - Discover each $R(s, a, s')$ when we experience the transition
- Step 2: Solve the learned MDP
 - Use, e.g., value or policy iteration, as before



Example: Model-Based Learning

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

Learned Model

$T(s,a,s')$

$T(B, \text{east}, C) = 1.00$
 $P(C, \text{east}, D) = 0.75$
 $P(C, \text{east}, A) = 0.25$
...

$R(s,a,s')$

$R(B, \text{east}, C) = -1$
 $R(C, \text{east}, D) = -1$
 $R(D, \text{exit}, x) = +10$
...

Pros and cons

- Pro:

- Makes efficient use of experiences (low *sample complexity*)

- Con:

- May not scale to large state spaces
 - Solving MDP is intractable for very large $|S|$
- RL feedback loop tends to magnify small model errors
- Much harder when the environment is partially observable

Basic idea of model-free methods

- To approximate expectations with respect to a distribution, you can either
 - ✓ Estimate the distribution from samples, compute an expectation
 - ✓ Or, bypass the distribution and estimate the expectation from samples directly

Example: Expected Age

Goal: Compute expected age of AI students

Known $P(A)$

$$E[A] = \sum_a P(a) \cdot a = 0.35 \times 20 + \dots$$

Without $P(A)$, instead collect samples $[a_1, a_2, \dots, a_N]$

“Model Based”: estimate $P(A)$:

$$\hat{P}(A=a) = N_a/N$$

$$E[A] \approx \sum_a \hat{P}(a) \cdot a$$

Why does this work? Because eventually you learn the right model.

“Model Free”: estimate expectation

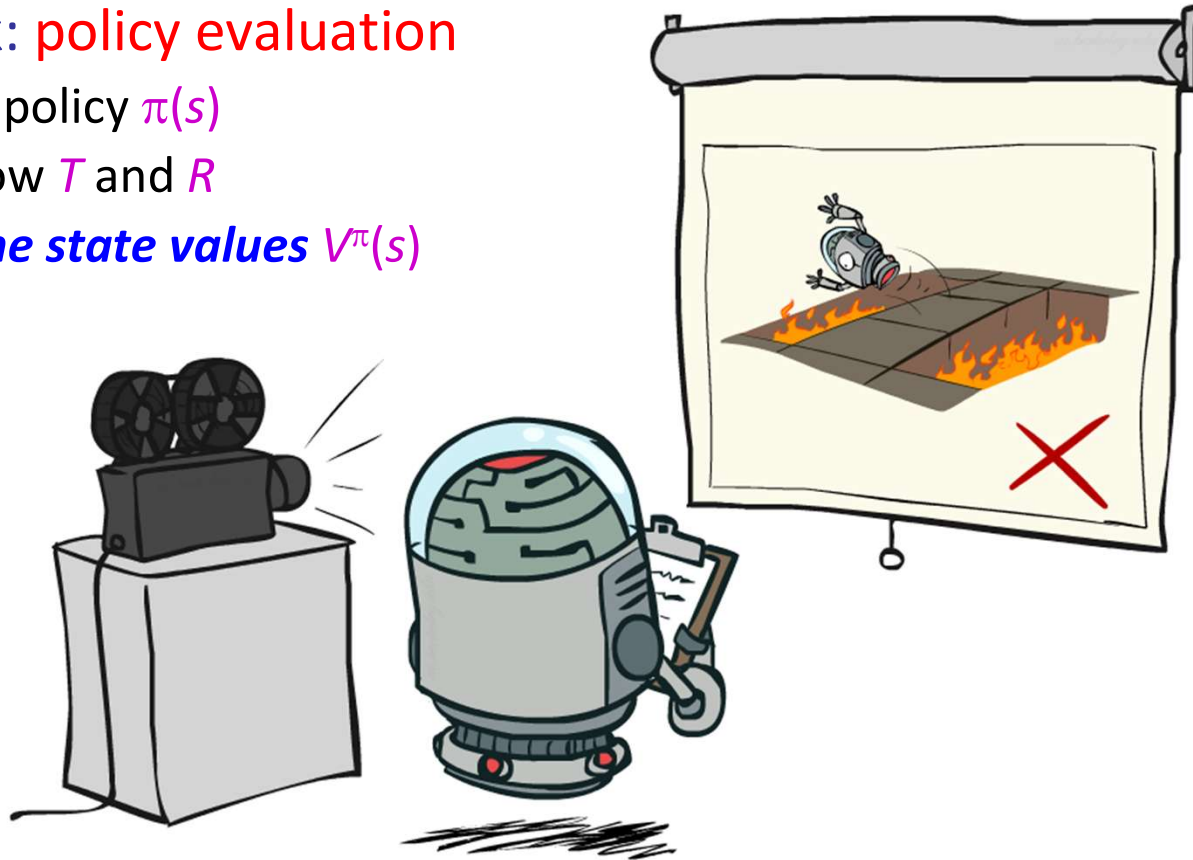
$$E[A] \approx 1/N \sum_i a_i$$

Why does this work? Because samples appear with the right frequencies.

Passive Reinforcement Learning

- Simplified task: **policy evaluation**

- Input: a fixed policy $\pi(s)$
- You don't know T and R
- **Goal: learn the state values $V^\pi(s)$**

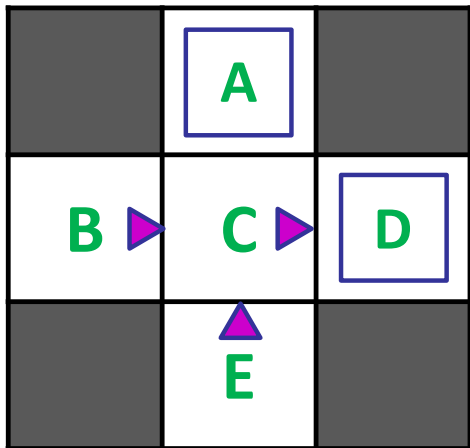


Direct evaluation (Monte Carlo learning)

- Goal: Estimate $V^\pi(s)$, i.e., expected total discounted reward from s onwards
- Idea:
 - Use *returns*, the actual sums of discounted rewards from s
 - Average over multiple trials and visits to s
- This is called **direct evaluation** (or direct utility estimation)

Example: Direct Estimation

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

Output Values

	-10	
+8 B	+4 C	+10 D
	-2 E	

Problems with Direct Estimation

- What's good about direct estimation?
 - It's easy to understand
 - It doesn't require any knowledge of T and R
 - It converges to the right answer in the limit
- What's bad about it?
 - Each state must be learned separately (fixable)
 - It **ignores information about state connections**
 - So, it takes a long time to learn

*E.g., B=at home, study hard
E=at library, study hard
C=know material, go to exam*

Output Values

	-10 A	
+8 B	+4 C	+10 D
	-2 E	

*If B and E both go to C
under this policy, how can
their values be different?*

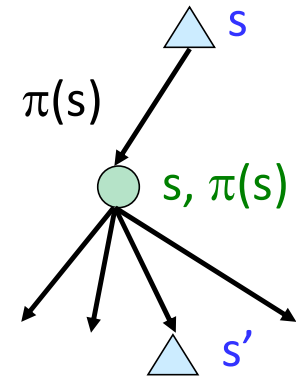
Temporal Difference Learning

- Big idea: learn from every experience!

- Update $V(s)$ each time we experience a transition (s, a, s', r)
- Likely outcomes s' will contribute updates more often

- Temporal difference learning of values

- Policy still fixed, still doing evaluation!
- Move values toward value of whatever successor occurs: running average



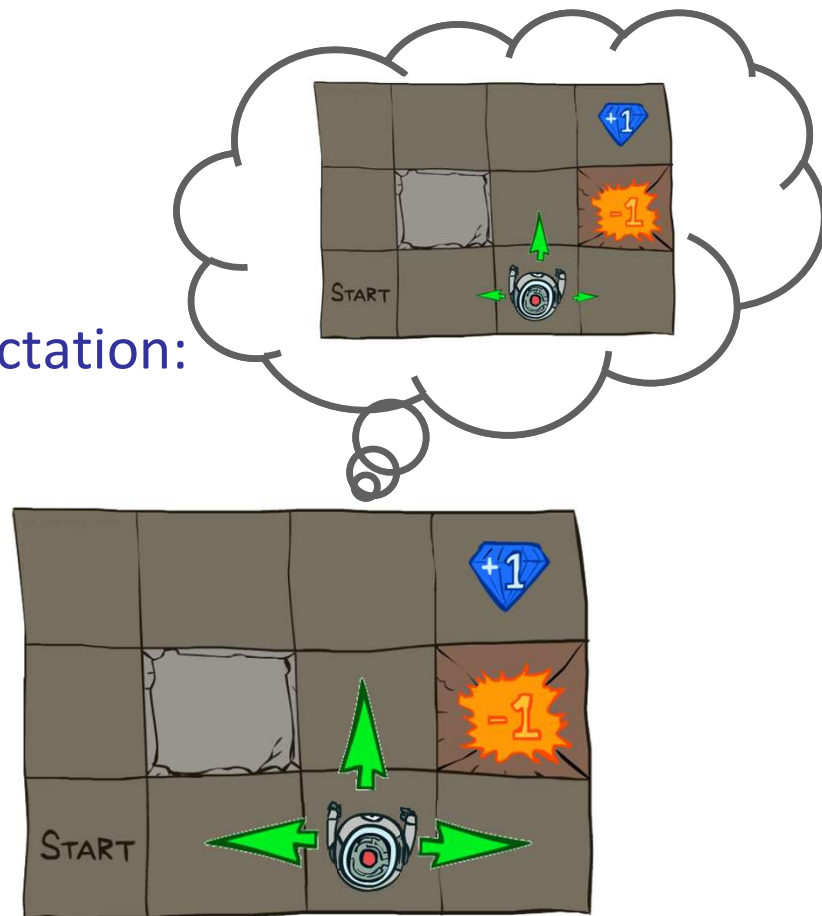
Sample of $V(s)$: $sample = R(s, \pi(s), s') + \gamma V^\pi(s')$

Update to $V(s)$: $V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + (\alpha)sample$

Same update: $V^\pi(s) \leftarrow V^\pi(s) + \alpha(sample - V^\pi(s))$

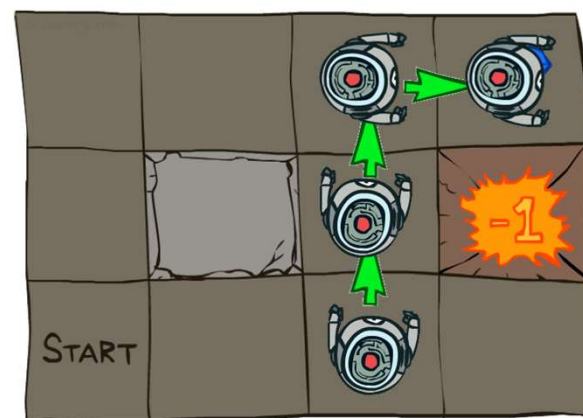
TD as approximate Bellman update

- Given a fixed policy, the value of a state is an expectation over next-state values:
 - $V^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')]]$
- Idea 1: Use actual samples to estimate the expectation:
 - $\text{sample}_1 = R(s, \pi(s), s_1') + \gamma V^\pi(s_1')$
 - $\text{sample}_2 = R(s, \pi(s), s_2') + \gamma V^\pi(s_2')$
 - ...
 - $\text{sample}_N = R(s, \pi(s), s_N') + \gamma V^\pi(s_N')$
 - $V^\pi(s) \leftarrow 1/N \sum_i \text{sample}_i$



TD as approximate Bellman update

- Idea 2: Update value of s after each transition s, a, s', r :
- Update $V^\pi([3,1])$ based on $R([3,1], \text{up}, [3,2])$ and $\gamma V^\pi([3,2])$
- Update $V^\pi([3,2])$ based on $R([3,2], \text{up}, [3,3])$ and $\gamma V^\pi([3,3])$
- Update $V^\pi([3,3])$ based on $R([3,3], \text{right}, [4,3])$ and $\gamma V^\pi([4,3])$



TD as approximate Bellman update

- Idea 3: Update values by maintaining a *running average*

“If one had to identify one idea as central and novel to RL, it would undoubtedly be TD learning.”



Richard Sutton

Running averages

- How do you compute the average of 1, 4, 7?
- Method 1: add them up and divide by N
 - $1+4+7 = 12$
 - $\text{average} = 12/N = 12/3 = 4$
- Method 2: keep a running average μ_n and a running count n
 - $n=0 \quad \mu_0=0$
 - $n=1 \quad \mu_1 = (0 \cdot \mu_0 + x_1)/1 = (0 \cdot 0 + 1)/1 = 1$
 - $n=2 \quad \mu_2 = (1 \cdot \mu_1 + x_2)/2 = (1 \cdot 1 + 4)/2 = 2.5$
 - $n=3 \quad \mu_3 = (2 \cdot \mu_2 + x_3)/3 = (2 \cdot 2.5 + 7)/3 = 4$
 - General formula: $\mu_n = ((n-1) \cdot \mu_{n-1} + x_n)/n$
 - $= [(n-1)/n] \mu_{n-1} + [1/n] x_n$ (weighted average of old mean, new sample)

Running averages contd.

- What if we use a weighted average with a fixed weight?
 - $\mu_n = (1-\alpha) \mu_{n-1} + \alpha x_n$
 - $n=1 \quad \mu_1 = x_1$
 - $n=2 \quad \mu_2 = (1-\alpha) \cdot \mu_1 + \alpha x_2 = (1-\alpha) \cdot x_1 + \alpha x_2$
 - $n=3 \quad \mu_3 = (1-\alpha) \cdot \mu_2 + \alpha x_3 = (1-\alpha)^2 \cdot x_1 + \alpha(1-\alpha)x_2 + \alpha x_3$
 - $n=4 \quad \mu_4 = (1-\alpha) \cdot \mu_3 + \alpha x_4 = (1-\alpha)^3 \cdot x_1 + \alpha(1-\alpha)^2 x_2 + \alpha(1-\alpha)x_3 + \alpha x_4$
- I.e., **exponential forgetting** of old values
- μ_n is a convex combination of sample values (weights sum to 1)
- $E[\mu_n]$ is a convex combination of $E[X_i]$ values, hence unbiased

TD as approximate Bellman update

- Idea 3: Update values by maintaining a *running average*
 - $\text{sample} = R(s, \pi(s), s') + \gamma V^\pi(s')$
 - $V^\pi(s) \leftarrow (1-\alpha) \cdot V^\pi(s) + \alpha \cdot \text{sample}$
 - $V^\pi(s) \leftarrow V^\pi(s) + \alpha \cdot [\text{sample} - V^\pi(s)]$
 - This is the *temporal difference learning rule*
 - $[\text{sample} - V^\pi(s)]$ is the “TD error”
 - α is the *learning rate*
- Observe a sample, move $V^\pi(s)$ a little bit to make it more consistent with its neighbor $V^\pi(s')$

Example: Temporal Difference Learning

States

	A	
B	C	D
	E	

Assume: $\gamma = 1$, $\alpha = 1/2$

Observed Transitions

B, east, C, -2

	0	
0	0	8
	0	

C, east, D, -2

	0	
-1	0	8
	0	

	0	
-1	3	8
	0	

$$V^\pi(s) \leftarrow (1-\alpha) V^\pi(s) + \alpha \cdot [R(s, \pi(s), s') + \gamma V^\pi(s')]$$

Temporal Difference Learning

- Passive setting (fixed policy π), like policy evaluation:

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')]]$$

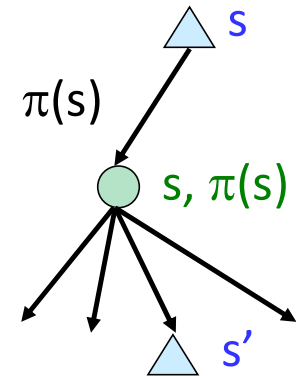
- Modifications:

1. Don't know T or R; estimate expectation from samples!

$$V^\pi(s) = \frac{1}{N} \sum_i [r_i + \gamma V^\pi(s'_i)]$$

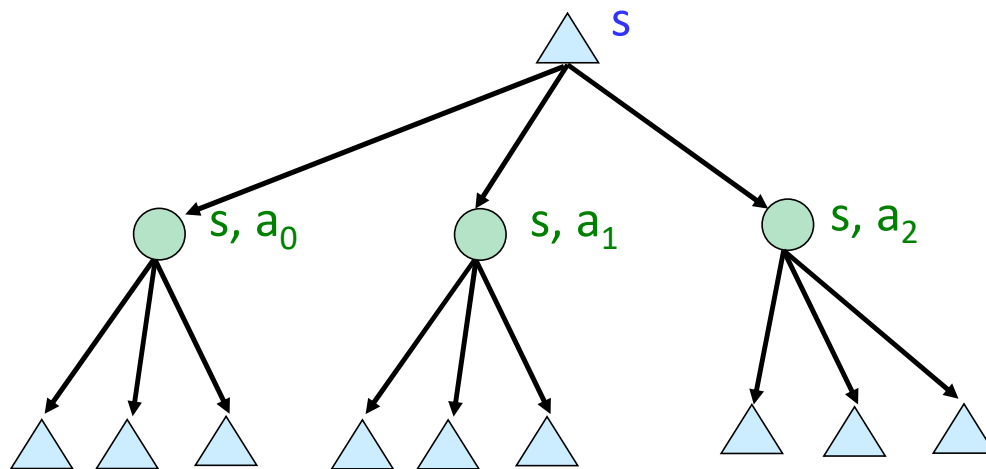
2. Update $V(s)$ after each transition (s, a, s', r) using running average.

3. Decay older samples as new ones come in.



Problems with TD Value Learning

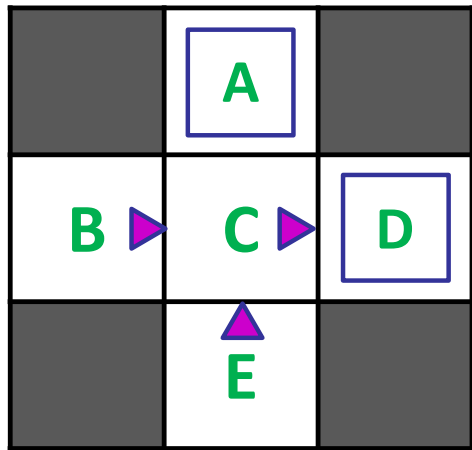
- Model-free policy evaluation! 🎉
- Bellman updates with running sample mean! 🎉



- Need the transition model to improve the policy! 🤖

Example: TD Value Estimation

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

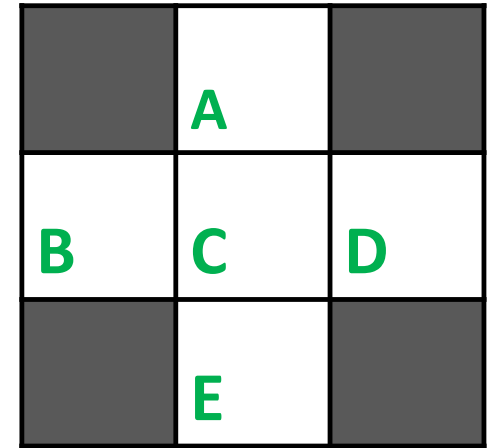
Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

Output Values



Example: TD Value Estimation

- Experience transition i : (s_i, a_i, s'_i, r_i) .
- Compute sampled value “target”: $r_i + \gamma V^\pi(s'_i)$.
- Compute “TD error”: $\delta_i = (r_i + \gamma V^\pi(s'_i)) - V^\pi(s_i)$.
- Update: $V^\pi(s_i) += \alpha_i \cdot \delta_i$.

B, east, C, -1
C, east, D, -1
D, exit, x, +10

B, east, C, -1
C, east, D, -1
D, exit, x, +10

E, north, C, -1
C, east, D, -1
D, exit, x, +10

E, north, C, -1
C, east, A, -1
A, exit, x, -10

s	V(s)
A	
B	
C	
D	
E	

i	s	a	s'	r	$r + \gamma V^\pi(s')$	$V^\pi(s)$	δ
1							
2							
3							
4							
5							
6							
7							

Example: TD Value Estimation

- Experience transition i : (s_i, a_i, s'_i, r_i) .
- Compute sampled value “target”: $r_i + \gamma V^\pi(s'_i)$.
- Compute “TD error”: $\delta_i = (r_i + \gamma V^\pi(s'_i)) - V^\pi(s_i)$.
- Update: $V^\pi(s_i) += \alpha_i \cdot \delta_i$.

B, east, C, -1
C, east, D, -1
D, exit, x, +10

B, east, C, -1
C, east, D, -1
D, exit, x, +10

E, north, C, -1
C, east, D, -1
D, exit, x, +10

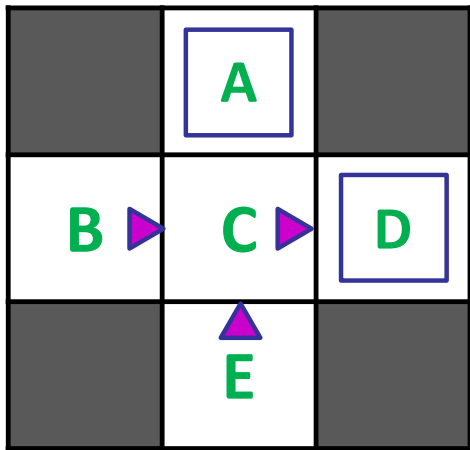
E, north, C, -1
C, east, A, -1
A, exit, x, -10

s	V(s)
A	0
B	-2
C	9
D	10
E	8

i	s	a	s'	r	$r + \gamma V^\pi(s')$	$V^\pi(s)$	δ
1	B	east	C	-1	$-1 + 0$	0	-1
2	C	east	D	-1	$-1 + 0$	0	-1
3	D	exit	---	10	$10 + 0$	0	+10
4	B	east	C	-1	$-1 + -1$	-1	-1
5	C	east	D	-1	$-1 + 10$	-1	+10
6	D	exit	---	10	$10 + 0$	10	0
7	E	north	C	-1	$-1 + 9$	0	+8

Example: TD Value Estimation

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

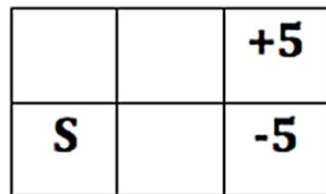
E, north, C, -1
C, east, A, -1
A, exit, x, -10

Output Values

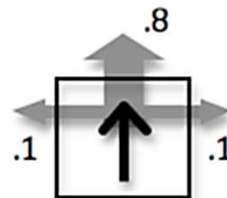
	-10	
+3	+4	+10
B	C	D
	+3	
	E	

In-Class Example 1

- This grid world MDP operates similar to the one we saw in class.
- The states are grid squares, identified by their row and column number (row first).
- The agent always starts in state (1,1), marked with the letter S.
- There are two terminal goal states, (2,3) with reward +5 and (1,3) with reward -5. Rewards are 0 in non-terminal states.
- The transition function is such that the intended agent movement (North, South, West, or East) happens with probability .8. With probability .1 each, the agent ends up in one of the states perpendicular to the intended direction.
- If a collision with a wall happens, the agent stays in the same state.



(a)



(b)

-cont-

1. Draw the optimal policy for this grid?
2. Suppose the agent knows the transition probabilities. Give the first two rounds of value iteration updates for each state, with a discount of 0.9. (Assume V_0 is 0 everywhere and compute V_i for times $i = 1, 2$).
3. Suppose the agent does not know the transition probabilities. What does it need to be able to do (or have available) in order to learn the optimal policy?
4. The agent starts with the policy that always chooses to go right, and executes the following three trials:
 - 1) (1,1)–(1,2)–(1,3),
 - 2) (1,1)–(1,2)–(2,2)–(2,3), and
 - 3) (1,1)–(2,1)–(2,2)–(2,3).

What are the monte carlo (direct evaluation) estimates for states (1,1) and (2,2), given these traces?

5. Using a learning rate of .1 and assuming initial values of 0, what updates does the TD-learning agent make after trials 1 and 2, above?

solution

■ 1.

S =	(1,1)	(1,2)	(1,3)	(2,1)	(2,2)	(2,3)
$\pi^*(S) =$	Up	Left	NA	Right	Right	NA

■ 2.

S =	(1,1)	(1,2)	(1,3)	(2,1)	(2,2)	(2,3)
$V_0(S) =$	0	0	0	0	0	0
$V_1(S) =$	0	0	0	0	$0.8 \times 5.0 = 4.0$	0
$V_2(S) =$	0	$0.9 \times 0.8 \times 4 + 0.1 \times -5 = 2.38$	0	$0.8 \times 0.9 \times 4.0 = 2.88$	$0.8 \times 5.0 = 4.0$	0

- 3. The agent must be able to explore the world by taking actions and observing the effects.
- 4. To compute the estimates, average the rewards received in the trajectories that went through the indicates states.
 - $V((1, 1)) = (-5 + 5 + 5)/3 = 5/3 = 1.666$
 - $V((2, 2)) = (5 + 5)/2 = 5$

-cont solution-

■ 5.

The general TD-learning update is:

$$V(s) = V(s) + \alpha(r + \gamma V(s') - V(s))$$

After trial 1, all of the updates will be zero, expect for:

$$V((1,2)) = 0 + .1(-5 + 0.9 \times 0 - 0) = -0.5$$

After trial 2, the updates will be:

$$V((1,1)) = 0 + .1(0 + 0.9 \times -0.5 - 0) = -0.045$$

$$V((1,2)) = -0.5 + .1(0 + 0.9 \times 0 + 0.5) = -0.45$$

$$V((2,2)) = 0 + .1(5 + 0.9 \times 0 - 0) = 0.5$$

SARSA (State-Action-Reward-State-Action)

- TD Needs the transition model to **improve the policy!**
- So....
- Idea: Learn the action-value function $Q(s, a)$ directly!
- SARSA uses the next action a_{t+1} chosen by the same policy
- $Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1})]$

Detour: Q-Value Iteration

- Value iteration: find successive (depth-limited) values

- Start with $V_0(s) = 0$, which we know is right
- Given V_k , calculate the depth $k+1$ values for all states:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

- But Q-values are more useful, so compute them instead

- Start with $Q_0(s,a) = 0$, which we know is right
- Given Q_k , calculate the depth $(k+1)$ q-values for all q-states:

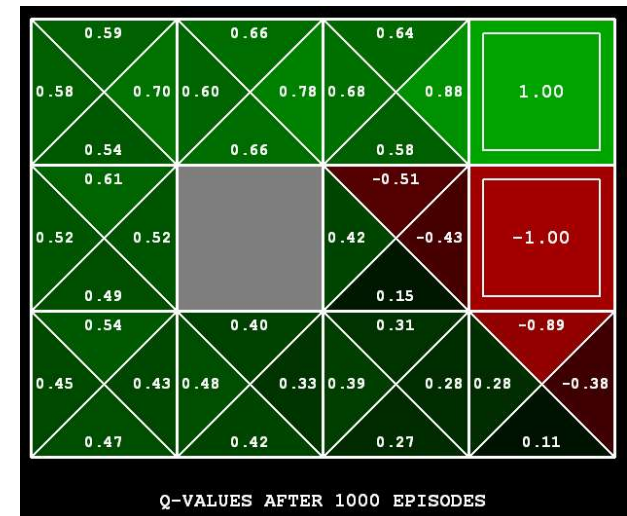
$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q_k(s', a')]$$

Q-learning as approximate Q-iteration

- Recall the definition of Q values:
 - $Q^*(s,a)$ = expected return from doing a in s and then behaving optimally thereafter; and $\pi^*(s) = \max_a Q^*(s,a)$
- Bellman equation for Q values:
 - $Q^*(s,a) = \sum_{s'} T(s,a,s') [R(s,a,s') + \gamma \max_{a'} Q^*(s',a')]]$
- Approximate Bellman update for Q values:
 - $Q(s,a) \leftarrow (1-\alpha) \cdot Q(s,a) + \alpha \cdot [R(s,a,s') + \gamma \max_{a'} Q(s',a')]]$
- We obtain a policy from learned $Q(s,a)$, with no model!
 - (No free lunch: $Q(s,a)$ table is $|A|$ times bigger than $V(s)$ table)

Q-Learning

- Learn $Q(s,a)$ values as you go
 - Receive a sample (s,a,s',r)
 - Consider your old estimate: $Q(s,a)$
 - Consider your new sample estimate:
 $sample = R(s,a,s') + \gamma \max_{a'} Q(s',a')$
- Incorporate the new estimate into a running average:
 $Q(s,a) \leftarrow (1-\alpha) Q(s,a) + \alpha \cdot [sample]$



Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

Q-Learning Properties

- Amazing result: Q-learning converges to optimal policy -- even if samples are generated from a suboptimal policy!
- This is called **off-policy learning**
- Caveats:
 - You have to explore enough
 - You have to eventually make the learning rate small enough
 - ... but not decrease it too quickly
 - Basically, in the limit, it doesn't matter how you select actions (!)



In-Class Example 2

- Imagine an unknown environment with four states (A, B, C, and X), two actions ($\leftarrow$ and $\rightarrow$). An agent acting in this environment has recorded the following episode:

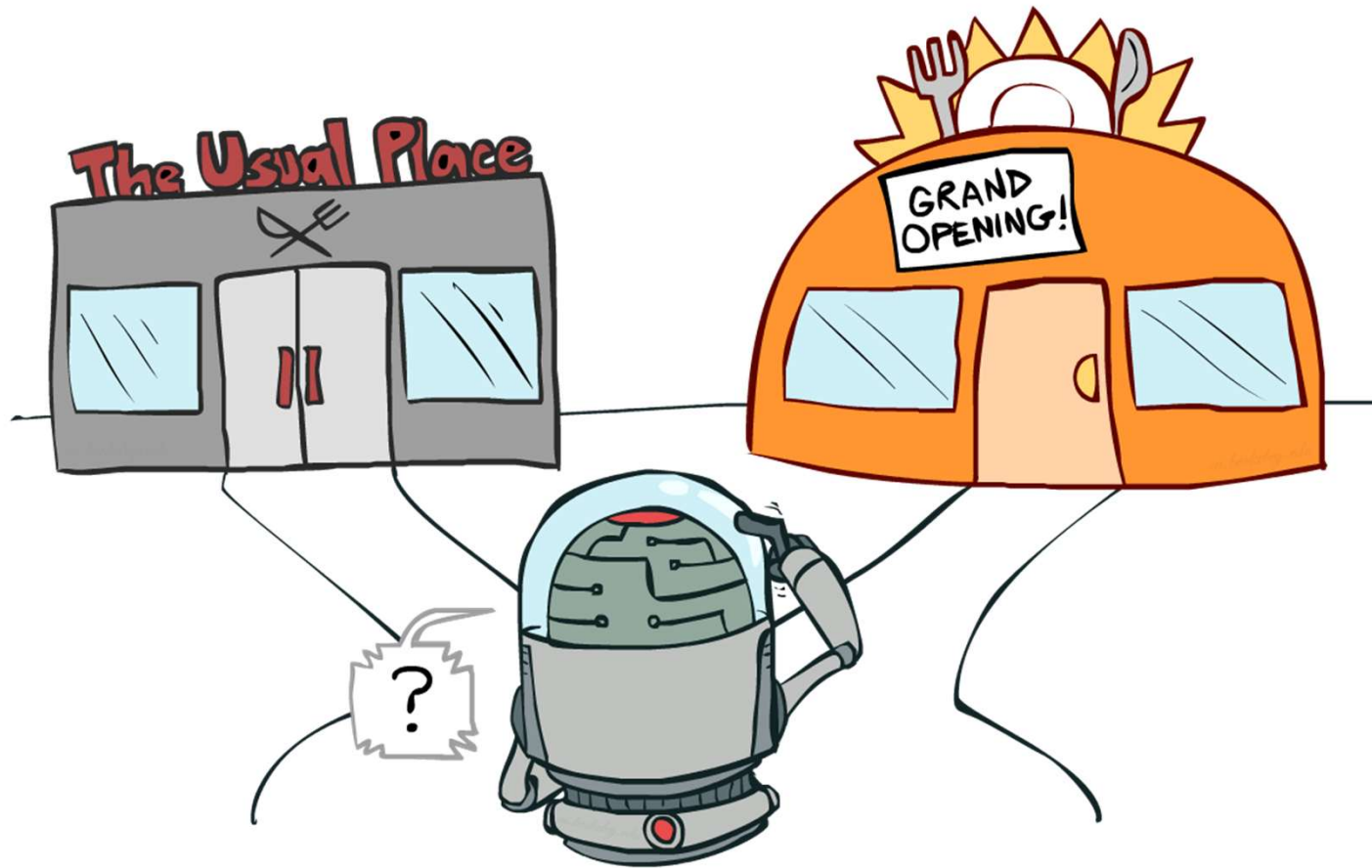
s	a	s'	r	Q-learning iteration numbers (for part b)
A	$\rightarrow$	B	0	1, 10, 19, ...
B	$\rightarrow$	C	0	2, 11, 20, ...
C	$\leftarrow$	B	0	3, 12, 21, ...
B	$\leftarrow$	A	0	4, 13, 22, ...
A	$\rightarrow$	B	0	5, 14, 23, ...
B	$\rightarrow$	A	0	6, 15, 24, ...
A	$\rightarrow$	B	0	7, 16, 25, ...
B	$\rightarrow$	C	0	8, 17, 26, ...
C	$\rightarrow$	X	1	9, 18, 27, ...

- a) Consider running model-based reinforcement learning based on the episode above. Calculate the following quantities:
- $T(B, \rightarrow, C) = ?$
 - $R(C, \rightarrow, X) = ?$

-cont-

- b) Now consider running Q-learning, repeating the above series of transitions in an infinite sequence. Each transition is seen at multiple iterations of Q-learning, with iteration numbers shown in the table above. After which iteration of Q-learning do the following quantities first become nonzero? (If they always remain zero, write never).
- $Q(A, \rightarrow)$?
 - $Q(B, \leftarrow)$?

Exploration vs. Exploitation



Exploration vs. Exploitation

- **Exploration**: try new things
- **Exploitation**: do what's best given what you've learned so far
- Key point: pure exploitation often gets **stuck in a rut** and never finds an optimal policy!

Exploration method: ϵ -greedy

- ϵ -greedy exploration

- Every time step, flip a biased coin
- With (small) probability ϵ , act randomly
- With (large) probability $1-\epsilon$, act on current policy

- Properties of ϵ -greedy exploration

- Every s,a pair is tried infinitely often
- Does a lot of stupid things
 - Jumping off a cliff *lots of times* to make sure it hurts
- Keeps doing stupid things for ever
 - Decay ϵ towards 0



In-class Example 3

Consider an unknown MDP with three states (A , B and C) and two actions ($\leftarrow$ and $\rightarrow$). Suppose the agent chooses actions according to some policy π in the unknown MDP, collecting a dataset consisting of samples (s, a, s', r) representing taking action a in state s resulting in a transition to state s' and a reward of r .

s	a	s'	r
A	$\rightarrow$	B	2
C	$\leftarrow$	B	2
B	$\rightarrow$	C	-2
A	$\rightarrow$	B	4

You may assume a discount factor of $\gamma = 1$.

Assume that all Q -values are initialized to 0, and use a learning rate of $\alpha = 1/2$

-
1. Run Q -learning on the above experience table and fill in the following Q -values:
 - $Q(A, \rightarrow) =$
 - $Q(B, \rightarrow) =$
 2. After running Q -learning and producing the above Q -values, you construct a policy π that maximizes the Q -value in a given state: $\pi = \arg \max_a Q(s, a)$. What are the actions chosen by the policy in states A and B ?
 3. In the limit of infinite timesteps, under which of the following exploration policies is Q -learning guaranteed to converge to the optimal Q -values for all state? (You may assume the learning rate α is chosen appropriately.)
 - a) A fixed policy taking actions uniformly at random.
 - b) A greedy policy.
 - c) An ϵ -greedy policy
 - d) A fixed optimal policy

solution

1.

$$Q_1(A, \rightarrow) = \frac{1}{2} \cdot Q_0(A, \rightarrow) + \frac{1}{2} \left(2 + \gamma \max_{a'} Q(B, a') \right) = 1$$
$$Q_1(C, \leftarrow) = 1$$
$$Q_1(B, \rightarrow) = \frac{1}{2}(-2 + 1) = -\frac{1}{2}$$
$$Q_2(A, \rightarrow) = \frac{1}{2} \cdot 1 + \frac{1}{2} \left(4 + \max_{a'} Q_1(B, a') \right)$$
$$= \frac{1}{2} + \frac{1}{2}(4 + 0) = \frac{5}{2}.$$

2. A: right, B: left

3. (i) and (iii)

For Q -learning to converge, every state-action pair (s, a) must occur infinitely often. A uniform random policy will achieve this in an ergodic MDP. A fixed optimal policy will not take any suboptimal actions and so will not explore enough. Similarly a greedy policy will stop taking actions the current Q -values suggest are suboptimal, and so will never update the Q -values for supposedly suboptimal actions.

In-class Example 4

Pacman is in an unknown MDP where there are three states [A, B, C] and two actions [Stop, Go]. We are given the following samples generated from taking actions in the unknown MDP. For the following problems, assume $\gamma = 1$ and $\alpha = 0.5$.

(a) We run Q-learning on the following samples:

s	a	s'	r
A	Go	B	2
C	Stop	A	0
B	Stop	A	-2
B	Go	C	-6
C	Go	A	2
A	Go	A	-2

What are the estimates for the following Q-values as obtained by Q-learning? All Q-values are initialized to 0.

(i) $Q(C, \text{Stop}) = ?$

(ii) $Q(C, \text{Go}) = ?$

solution

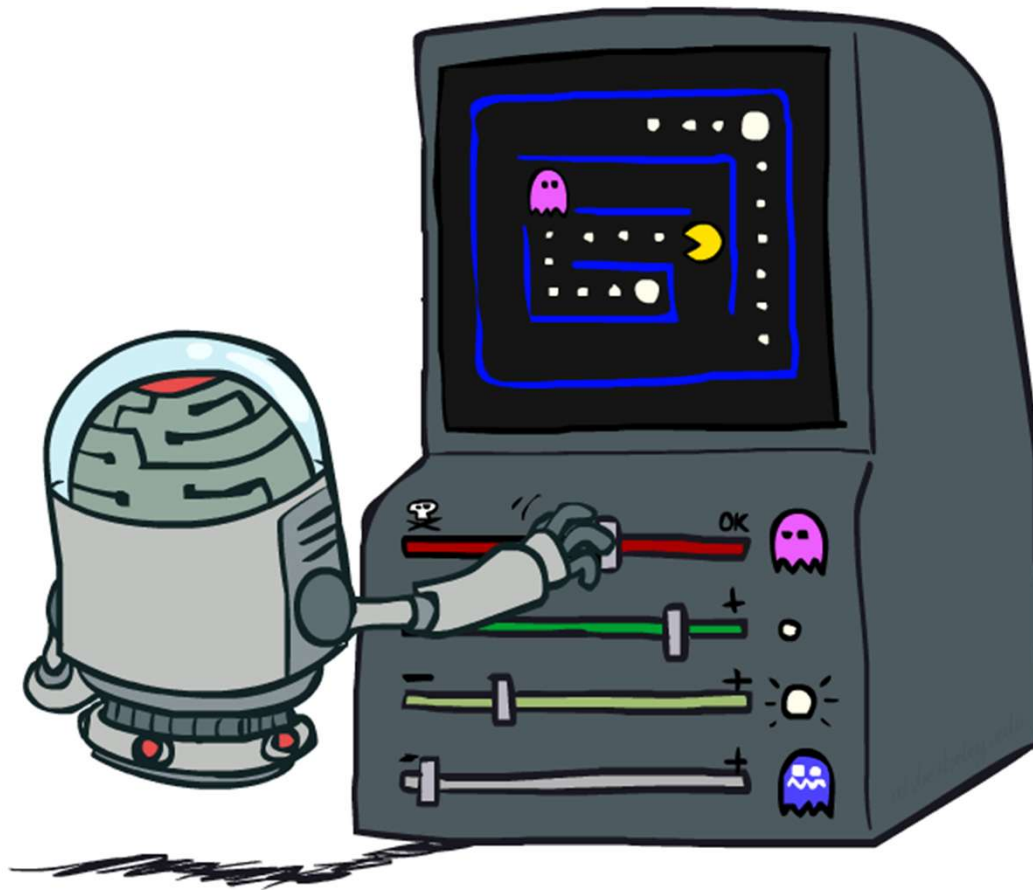
For this, we only need to consider the following three samples.

$$Q(A, Go) \leftarrow (1 - \alpha)Q(A, Go) + \alpha(r + \gamma \max_a Q(B, a)) = 0.5(0) + 0.5(2) = 1$$

$$Q(C, Stop) \leftarrow (1 - \alpha)Q(C, Stop) + \alpha(r + \gamma \max_a Q(A, a)) = 0.5(0) + 0.5(1) = 0.5$$

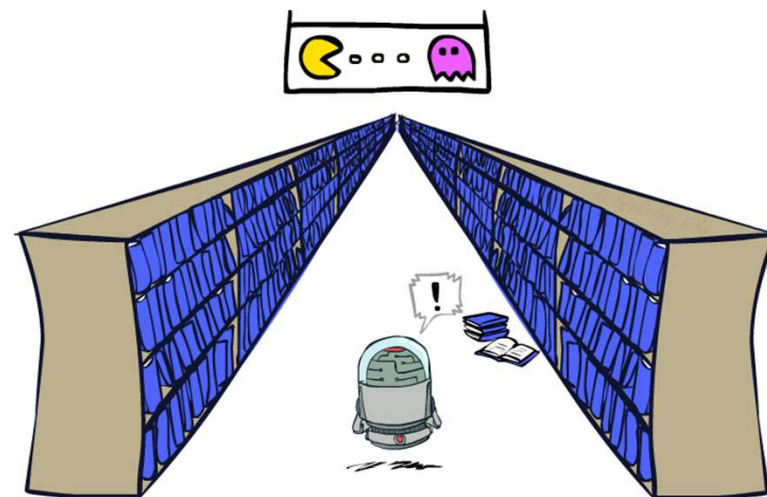
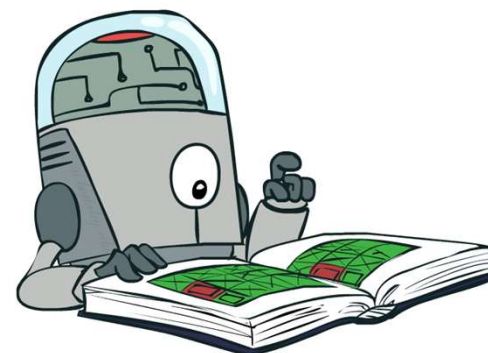
$$Q(C, Go) \leftarrow (1 - \alpha)Q(C, Go) + \alpha(r + \gamma \max_a Q(A, a)) = 0.5(0) + 0.5(3) = 1.5$$

Approximate Q-Learning



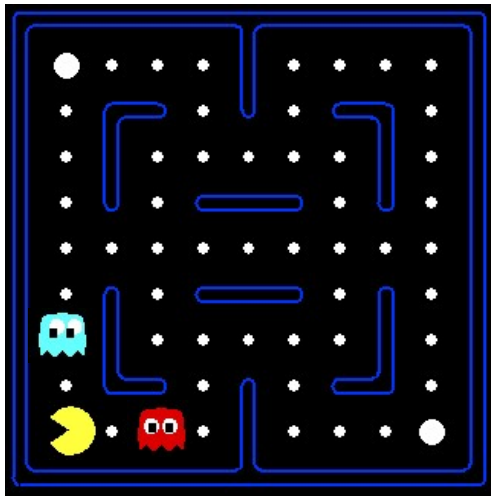
Generalizing Across States

- Basic Q-Learning keeps a table of all Q-values
- In realistic situations, we cannot possibly learn about **every single state!**
 - Too many states to visit them all in training
 - Too many states to hold the Q-tables in memory
- Instead, we want to generalize:
 - Learn about some small number of training states from experience
 - Generalize that experience to new, similar situations
 - Can we apply some machine learning tools to do this?

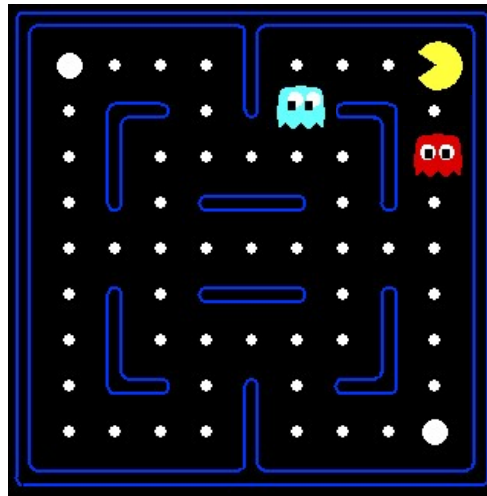


Example: Pacman

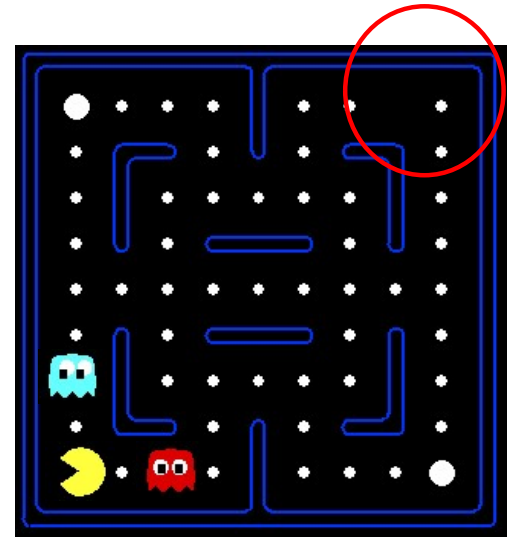
Let's say we discover through experience that this state is bad:



In naïve q-learning, we know nothing about this state:



Or even this one!



Feature-Based Representations

- Solution: describe a state using a vector of features
 - Features are functions from states to real numbers (often 0/1) that capture important properties of the state
 - Example features:
 - Distance to closest ghost f_{GST}
 - Distance to closest dot
 - Number of ghosts
 - $1 / (\text{distance to closest dot})$ f_{DOT}
 - Is Pacman in a tunnel? (0/1)
 - etc.
 - Can also describe a q-state (s, a) with features (e.g., action moves closer to food)



Linear Value Functions

- We can express V and Q (approximately) as weighted linear functions of feature values:
 - $V_w(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$
 - $Q_w(s,a) = w_1 f_1(s,a) + w_2 f_2(s,a) + \dots + w_n f_n(s,a)$
- Advantage: our experience is summed up in a few powerful numbers
 - Can compress a value function for chess (10^{43} states) down to about 30 weights!
- Disadvantage: states may share features but have very different expected utility!

Updating a linear value function

- Original Q-learning rule tries to reduce prediction error at s,a :
 - $Q(s,a) \leftarrow Q(s,a) + \alpha \cdot [R(s,a,s') + \gamma \max_{a'} Q(s',a') - Q(s,a)]$
- Instead, we update the weights to try to reduce the error at s,a :
 - $w_i \leftarrow w_i + \alpha \cdot [R(s,a,s') + \gamma \max_{a'} Q(s',a') - Q(s,a)] \partial Q_w(s,a) / \partial w_i$
 $= w_i + \alpha \cdot [R(s,a,s') + \gamma \max_{a'} Q(s',a') - Q(s,a)] f_i(s,a)$
- Intuitive interpretation:
 - Adjust weights of active features
 - If something bad happens, blame the features we saw; decrease value of states with those features. If something good happens, increase value!

Why?

Q-learning with function approximation (e.g., linear or neural net) minimizes the TD error squared. The weights are updated to reduce:

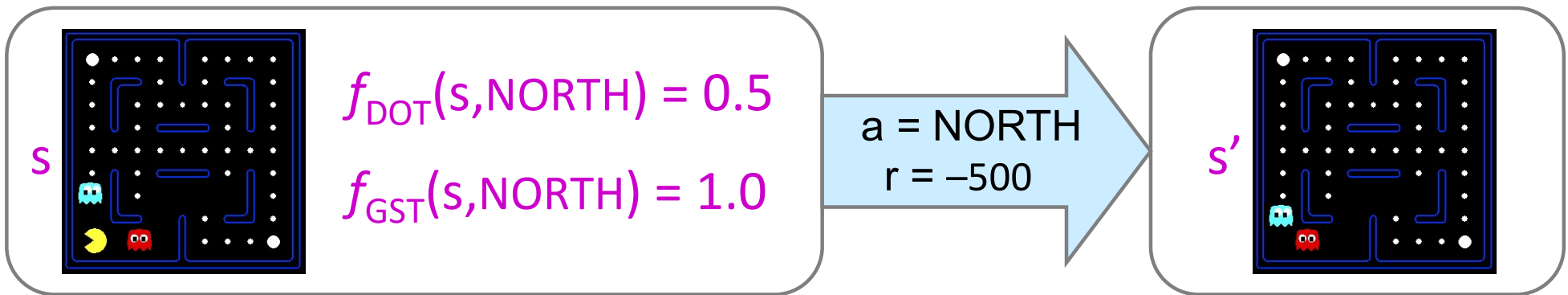
$$\frac{1}{2}(\text{Target} - \text{Prediction})^2 = \frac{1}{2} \left(R(s, a, s') + \gamma \max_{a'} Q(s', a') - Q_w(s, a) \right)^2$$
$$\min_w \frac{1}{2} \left(R(s, a, s') + \gamma \max_{a'} Q(s', a') - (w_1 f_1(s, a) + w_2 f_2(s, a) + \dots + w_n f_n(s, a)) \right)^2$$

Using gradient descent, the update rule becomes:

$$w_1 \leftarrow w_1 - \alpha \frac{\partial \frac{1}{2}(\text{Target} - \text{Prediction})^2}{\partial w_1}$$
$$w_1 = w_1 - \alpha \left(-\frac{1}{2} \times 2 \times f_1(s, a) \times (\text{Target} - \text{Prediction}) \right)$$
$$w_1 = w_1 + \alpha f_1 \left(R(s, a, s') + \gamma \max_{a'} Q(s', a') - Q_w(s, a) \right)$$

Example: Q-Pacman

$$Q(s,a) = 4.0 f_{\text{DOT}}(s,a) - 1.0 f_{\text{GST}}(s,a)$$



$$f_{\text{DOT}}(s, \text{NORTH}) = 0.5$$

$$f_{\text{GST}}(s, \text{NORTH}) = 1.0$$

$$Q(s, \text{NORTH}) = +1$$

$$r + \gamma \max_{a'} Q(s', a') = -500 + 0$$

$$Q(s', \cdot) = 0$$

difference = -501



$$w_{\text{DOT}} \leftarrow 4.0 + \alpha[-501]0.5$$

$$w_{\text{GST}} \leftarrow -1.0 + \alpha[-501]1.0$$

$$Q(s,a) = 3.0 f_{\text{DOT}}(s,a) - 3.0 f_{\text{GST}}(s,a)$$

In-Class Example 5

- Pacman is in an unknown MDP where there are three states [A, B, C] and two actions [Stop, Go]. For the following problem, assume $\gamma = 1$ and $\alpha = 0.5$.
- we will switch to a feature based representation. We will use two features:

- $f_1(s, a) = 1$
- $f_2(s, a) = \begin{cases} 1 & a = \text{Go} \\ -1 & a = \text{Stop} \end{cases}$

- Starting from initial weights of 0, compute the updated weights after observing the following samples:

s	a	s'	r
A	Go	B	4
B	Stop	A	0

- i. What are the weights after the first update? (using the first sample)
- ii. What are the weights after the second update? (using the second sample)

Solution

(i)

$$Q(A, Go) = w_1 f_1(A, Go) + w_2 f_2(A, Go) = 0$$
$$difference = [r + \max_a Q(B, a)] - Q(A, Go) = 4$$
$$w_1 = w_1 + \alpha(difference) f_1 = 2$$
$$w_2 = w_2 + \alpha(difference) f_2 = 2$$

(ii)

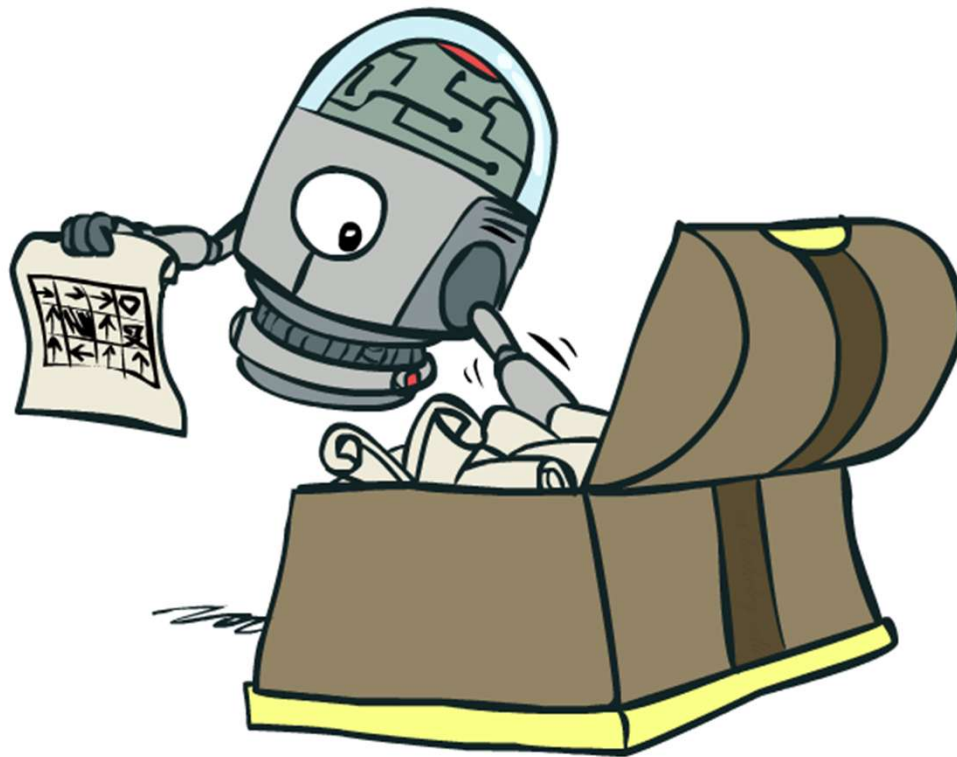
$$Q(B, Stop) = w_1 f_1(B, Stop) + w_2 f_2(B, Stop) = 2(1) + 2(-1) = 0$$
$$Q(A, Go) = w_1 f_1(A, Go) + w_2 f_2(A, Go) = 2(1) + 2(1) = 4$$
$$difference = [r + \max_a Q(A, a)] - Q(B, Stop) = [0 + 4] - 0 = 4$$
$$w_1 = w_1 + \alpha(difference) f_1 = 4$$
$$w_2 = w_2 + \alpha(difference) f_2 = 0$$

Approaches to reinforcement learning

1. Model-based: Learn the model, solve it, execute the solution
2. Learn values from experiences, use to make decisions
 - a. Direct evaluation (Monte Carlo Learning)
 - b. Temporal difference learning (TD learning)
 - c. SARSA
 - d. Q-learning
3. Optimize the policy directly

Policy Search

Not in exam



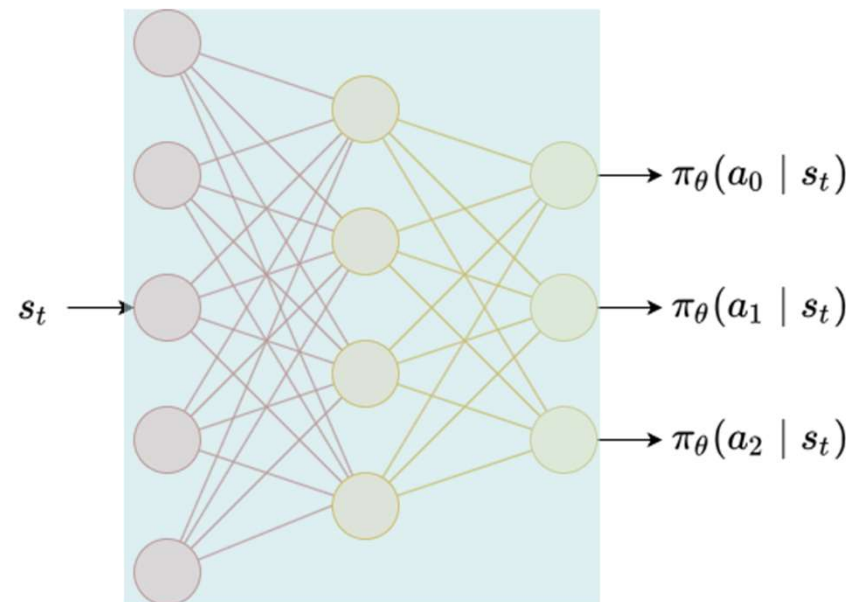
Intro

Not in exam

- Main focus of value-based methods: evaluate policies.
 - objective is to minimize a loss between predicted and target values
- In Policy-based methods, we model the policy π directly. We can parametrize our policy using some parameters θ to produce a distribution over actions:

$$\pi_{\theta}(a_t | s_t) = p(a_t | s_t, \theta)$$

- Main focus of policy-based methods: to maximize the performance of a parameterized policy
 - the performance of an agent is the expected total discounted reward from the initial state
 - In policy-based methods with stochastic policies, exploration is embedded in the learned function, and converging to a deterministic policy for a given state while training is possible.



Policy Gradient Loss Function

Not in exam

- The objective value for a certain policy with parameters θ is denoted $J(\theta)$:

$$J(\theta) = E_{\tau \sim \pi_{\theta}}[G(\tau)] = \sum_{\tau} p_{\theta}(\tau) G(\tau)$$

- Where:

- $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ is a trajectory.
 - $\pi_{\theta}(a | s)$ is the policy parameterized by θ
 - $G(\tau)$ is the cumulative reward of a trajectory: $G(\tau) = \sum_{t=0}^T \gamma^t r_t$
- We want to maximize this objective via **gradient ascent** on $J(\theta)$ with respect to policy parameters θ :

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

Computing Gradient

Not in exam

- $J(\theta) = E_{\tau \sim \pi_\theta}[G(\tau)] = \sum_{\tau} p_\theta(\tau) G(\tau)$
- **GOAL: computing $\nabla_\theta J(\theta)$**
- Using Log-derivative trick: $\nabla f(x) = f(x) \nabla \log(f(x))$
- $\nabla_\theta J(\theta) = \sum_{\tau} p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) G(\tau)$
- We can compute probability of a trajectory:

$$p(\tau) = p(s_0) \prod_{t=0}^T p(s_{t+1} | s_t, a_t) \pi_\theta(a_t | s_t)$$

Chain rule of probability:
 $P(A,B,C)=P(A)P(B|A)P(C|A,B)$

$$p(\tau) = p(s_0) p(a_0 | s_0) p(s_1 | s_0, a_0) p(a_1 | s_0, a_0, s_1) \cdots p(s_T | s_0, a_0, s_1, a_1, \dots, s_{T-1}, a_{T-1})$$

Note that: actions are selected probabilistically according to the policy π_θ

- The log-probability of a trajectory τ is then just:

$$\begin{aligned} \log p(\tau) &= \log \left(p(s_0) \prod_{t=0}^T p(s_{t+1} | s_t, a_t) \pi_\theta(a_t | s_t) \right) \\ &= \log p(s_0) + \sum_{t=0}^T (\log p(s_{t+1} | s_t, a_t) + \log \pi_\theta(a_t | s_t)) \end{aligned}$$

$$\nabla_\theta \log p_\theta(\tau) = \nabla_\theta \log p(s_0) + \sum_{t=0}^T (\nabla_\theta \log p(s_{t+1} | s_t, a_t) + \nabla_\theta \log \pi_\theta(a_t | s_t))$$

-cont-

Not in exam

- $\nabla_{\theta} \log p_{\theta}(\tau) = \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$
- $\nabla_{\theta} J(\theta) = \sum_{\tau} p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) G(\tau) = \sum_{\tau} p_{\theta}(\tau) (\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)) G(\tau)$
- But this is exactly the form of an expectation over τ :

$$\nabla_{\theta} J(\theta) = E_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) G(\tau) \right]$$

This is called Policy gradient

- ✓ For each action, we essentially make a gradient step that changes $\log \pi_{\theta}(a_t|s_t)$ based on the scale of $G(\tau)$
 - ✓ If we have a bad trajectory and $G(\tau) < 0$ then we decrease the log probability of each action along the trajectory
 - ✓ If we have a good trajectory and $G(\tau) > 0$ then we increase the log probability of each action along the trajectory

Reinforce Algorithm

Not in exam

- Why would I change the probability of an action at time t according to the reward seen at some earlier time $t' < t$?

$$\nabla_{\theta} J(\theta) = E_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G(t) \right]$$

$$\text{Where } G(t) = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

- We run a **rollout** of the policy π_{θ} in our environment for T time steps, storing the states, actions, rewards, and done flags at each time step t . Then, we compute an empirical estimate for the gradient:

$$\hat{g} = \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G(t)$$

- For autograd library like PyTorch by defining a loss function:

$$Loss = \sum_{t=0}^T \log \pi_{\theta}(a_t | s_t) G(t)$$

Reinforce Algorithm (Monte Carlo Policy Gradient)

Not in exam

1. Initialization:

- Initialize policy parameters θ (e.g., for a neural network policy).
- Set learning rate α .

2. Collect Trajectories:

- For each episode, interact with the environment using the current policy $\pi_\theta(a | s)$ to generate a trajectory:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T)$$

- T is the final timestep when the episode ends.

3. Compute Returns:

- Calculate the return $G(t)$ for each timestep t in the trajectory:

$$G(t) = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

4. Policy Gradient Estimation:

- For each timestep t , compute the policy gradient:

$$\nabla_\theta J(\theta) = E_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t | s_t) G(t)]$$

5. Update Parameters:

- Perform a gradient ascent step to maximize the expected return:

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

6. Repeat:

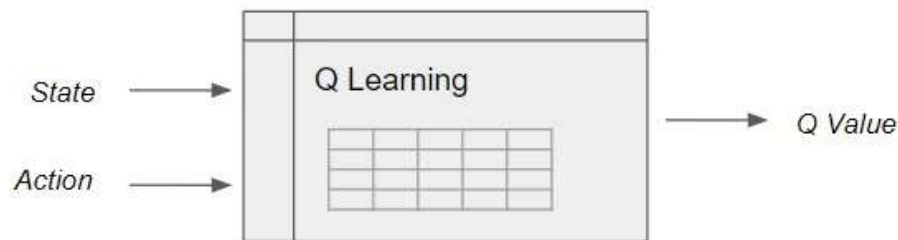
- Iterate over episodes, updating the policy parameters θ .

DQN

Not in exam

- Q-table can handle simple problems with few states

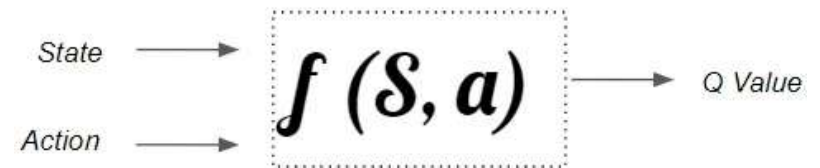
- Fundamentally, a Q-table maps state and action pairs to a Q-value.



- However, in a real-world scenario, the number of states could be huge, making it computationally intractable to build a table.

- Use a Q-Function for real-world problems

- use a function rather than a Q-table

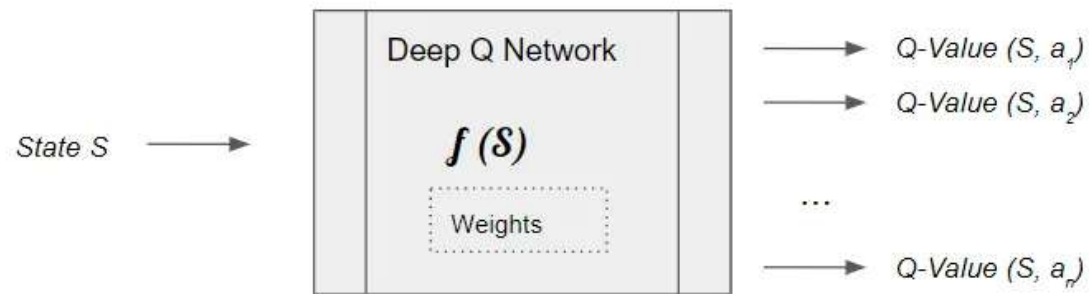


- Neural Nets are the best Function Approximators
 - neural networks are excellent at modeling complex functions, we can use a neural network, which we call a Deep Q Network, to estimate this Q function.

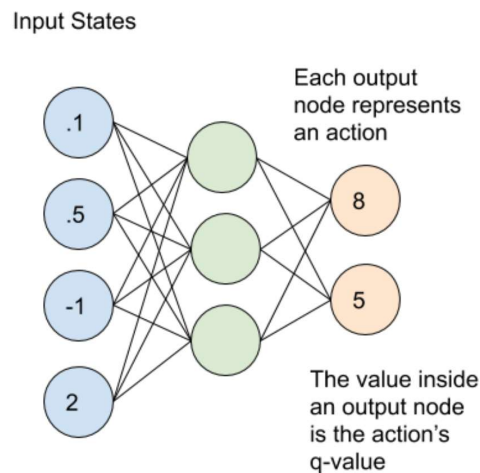
DQN

Not in exam

- maps a state to the Q values of all the actions that can be taken from that state.



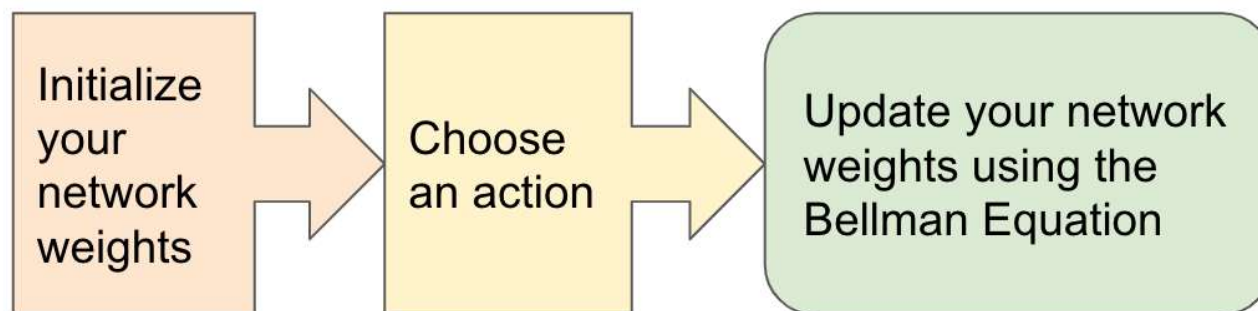
- It learns the network's parameters (weights) such that it can output the Optimal Q values.



DQN

Not in exam

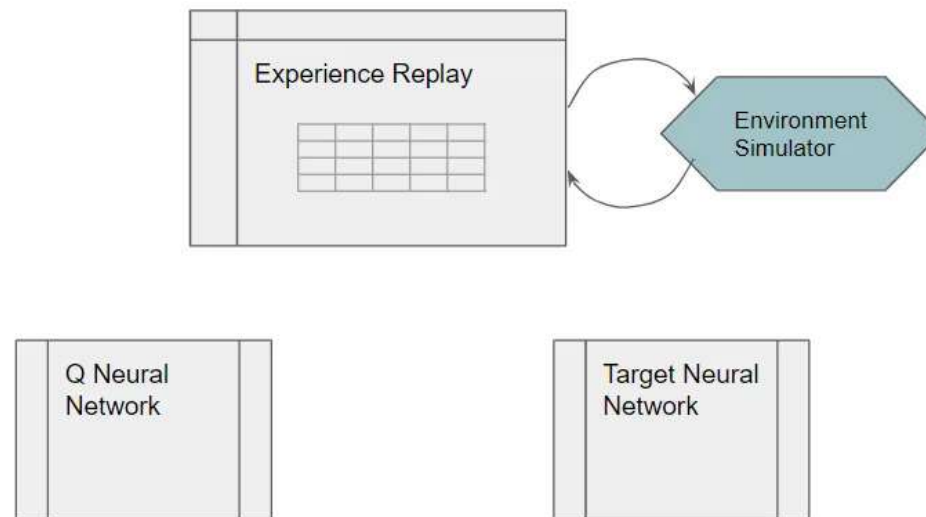
- The underlying principle of a Deep Q Network is very similar to the Q Learning algorithm.
 - It starts with arbitrary Q-value estimates
 - explores the environment using the ϵ -greedy policy.
 - Update your network weights using the Bellman Equation



DQN Architecture

Not in exam

- The DQN architecture has two neural nets:
 - Q network
 - Target networks,and a component called Experience Replay.

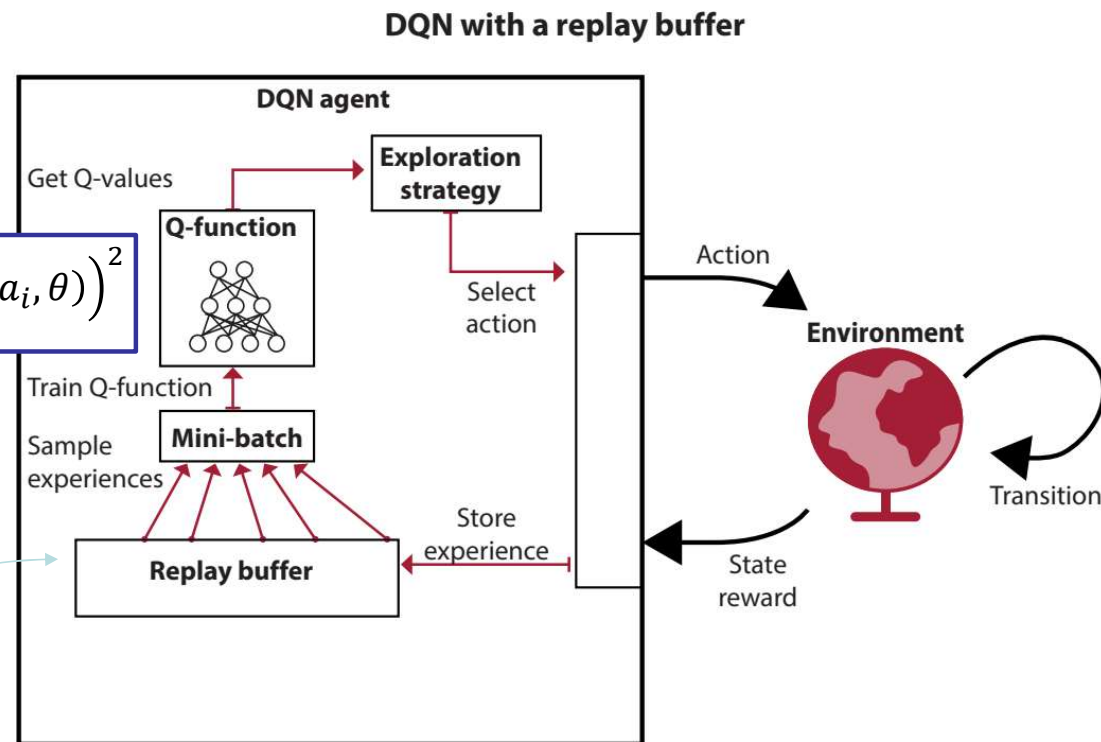
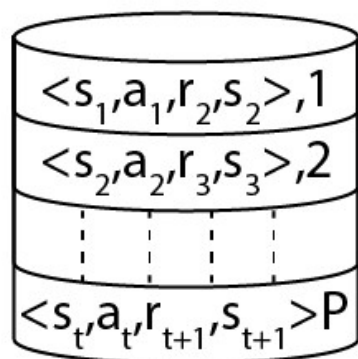


DQN with Replay Buffer

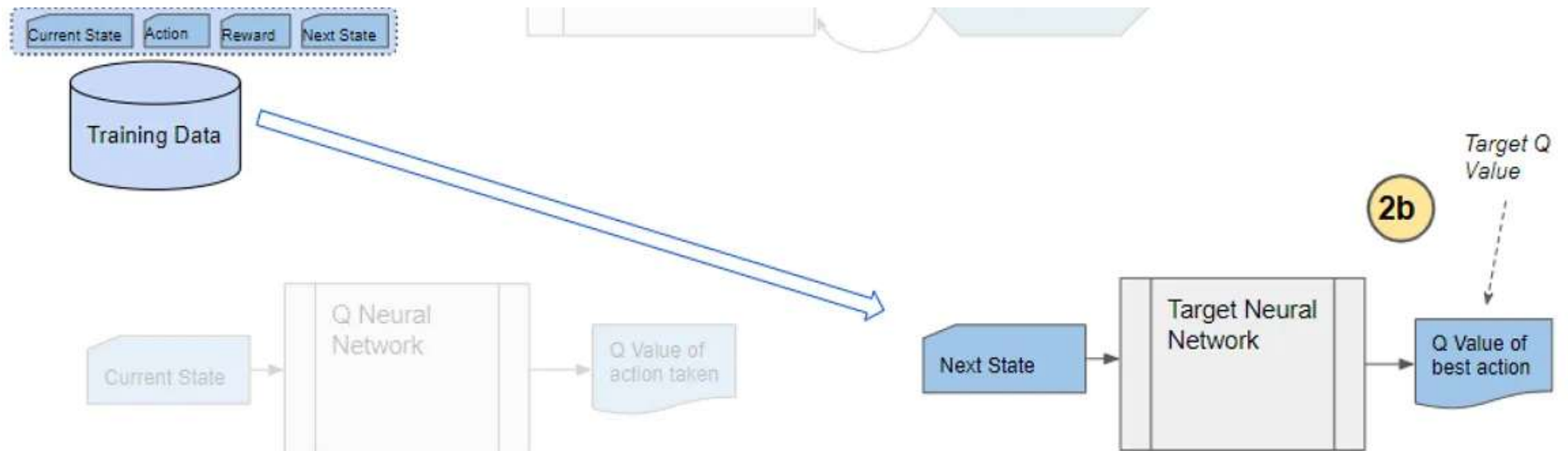
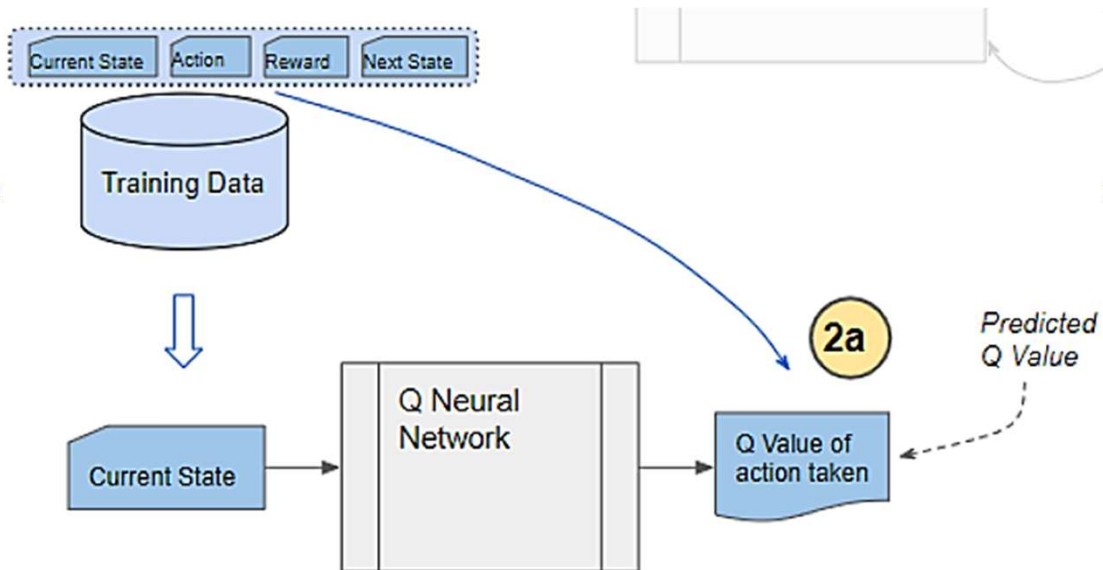
Not in exam

- Storing past transitions in replay buffer reduces the correlation between samples in a batch
- obtaining the experiences for training by sampling uniformly at random the replay buffer D , instead of using the online experiences as before

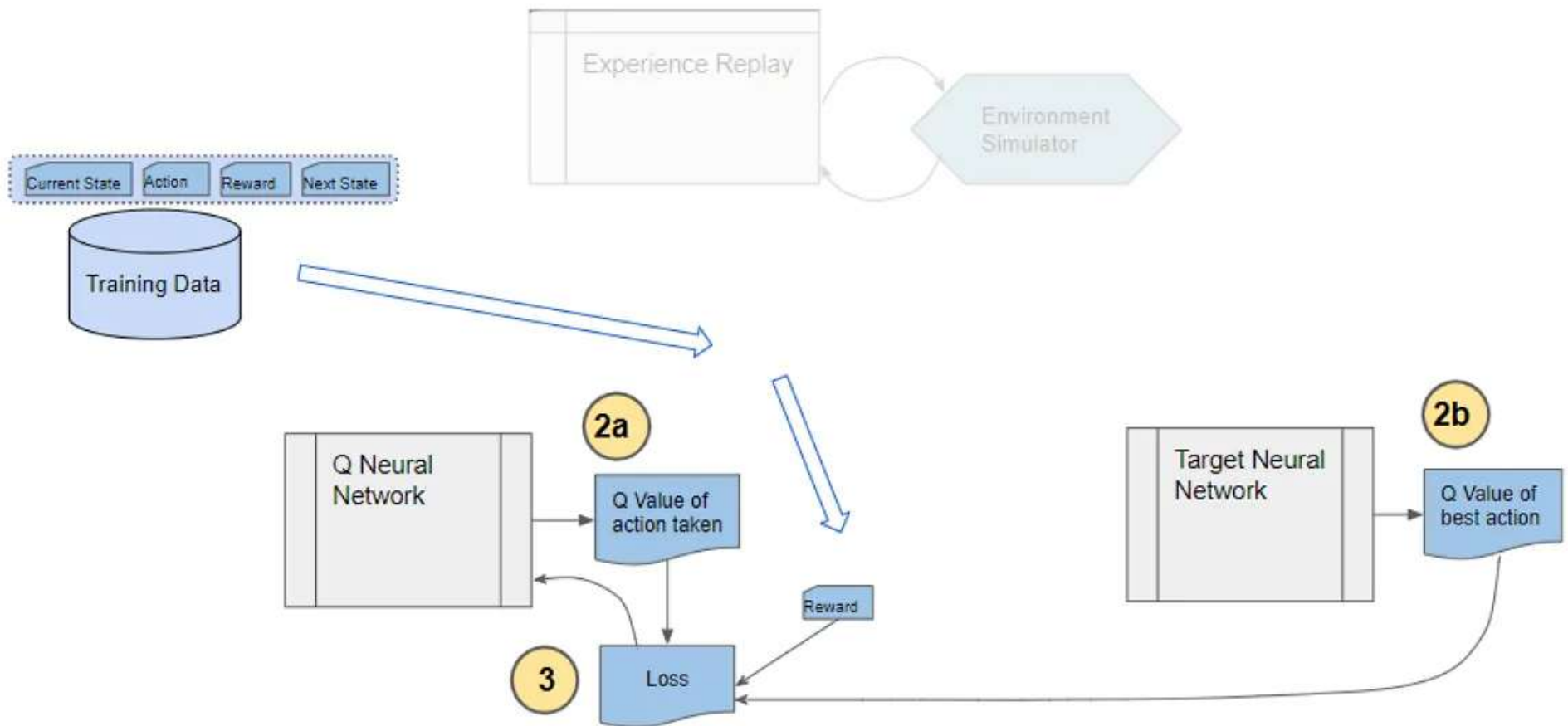
$$Loss = \sum_{i \in B} \left(r_i + \gamma \max_{a'} Q_{Target}(s'_i, a'; \theta^-) - Q_{predicted}(s_i, a_i, \theta) \right)^2$$



Not in exam



Not in exam



Why do we need Experience Replay?

- Sequential actions are highly correlated with one another and are not randomly shuffled, as the network would prefer.
- This ensures that the batch is 'shuffled' and contains enough diversity from older and newer samples to allow the network to learn weights that generalize to all the scenarios that it will be required to handle.

Why do we need a second neural network (Target Network)?

- The Q Network's weights get updated at each time step, which improves the prediction of the Predicted Q value. However, since the network and its weights are the same, it also changes the direction of our predicted Target Q values.
- They do not remain steady but can fluctuate after each update. This is like chasing a moving target
- It has been found that using a Target Network results in more stable training.

(1) Suppose we freeze the target for a few steps.



(2) That way, the optimizer can make stable progress toward it.



(3) We eventually update the target, and repeat.



(4) This allows the algorithm to make stable progress.



Summary

- RL solves MDPs via direct experience of transitions and rewards
- There are several approaches:
 - Learn the MDP model and solve it
 - Learn V directly from sums of rewards, or by TD local adjustments
 - Still need a model to make decisions by lookahead
 - Learn Q by local Q-learning adjustments, use it directly to pick actions
 - Optimize the policy directly
- Scaling up with feature representations and approximation